



Revealing unseen differences: A change metric perspective on deep learning software lifecycle for fine-tuning data debugging

Wenjie Xu , Yanhui Li *, Mingliang Ma , Lin Chen , Yuming Zhou

State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, 210023, China

ARTICLE INFO

Keywords:

Change metrics
Data debugging
Deep learning
Software measurement

ABSTRACT

As deep learning (DL) software becomes more widely employed, there is a growing demand for robust debugging techniques to enhance its quality and reliability. Due to the data-driven paradigm of DL software, data plays a fundamental role throughout its lifecycle, whose errors have received far less attention than program and model errors, which can also degrade model performance. Existing data debugging methodologies primarily focus on the software development phase, mainly overlooking the subsequent software iterations. Given the inherent characteristics of DL software, these iterative processes often include model fine-tuning, in which fine-tuning data would be a central element in determining the performance of fine-tuned DL software. Moreover, the fine-tuning datasets may be extracted from the application context with unavoidable noise or other interference factors that cause DL software defects. Despite the significance of fine-tuning data, there is currently little research specifically focused on the debugging of fine-tuning data.

In this paper, we propose a new set of change metrics based on the runtime information collected during pretraining and fine-tuning processes, which contains two aspects: Descriptive Statistics Difference Metrics (DSDM) and Multidimensional Data Comparison Metrics (MDCM). Our critical insight is that some changes in data captured during pretraining and fine-tuning processes may be related to bugs in the fine-tuning data. We build predictive models using five learners separately with our change metrics on six datasets and 233 DL models. The results indicate that (a) our change metrics are more effective than the baseline in fault diagnosis for fine-tuning data; (b) in most cases, using DSDM and MDCM together is more effective than using them separately; (c) the time cost of our approach primarily depends on the model training process, which is generally acceptable overall.

1. Introduction

Deep learning (DL) techniques have achieved considerable success across various domains, such as image recognition [1] and natural language processing [2], which causes DL software to become deeply entrenched in our daily lives. Due to the inherently stochastic nature [3], it is not feasible for DL software to produce correct outputs in every prediction. As the applications of DL software penetrate further into safety-critical fields such as medical diagnosis [4], autonomous driving [5], and credit assessment [6], they bring along emergent concerns regarding the quality and reliability of DL software [7–9], which necessitates developing innovative and more effective debugging techniques to resolve defects in DL software.

* Corresponding author.

E-mail addresses: wenjiexu@smail.nju.edu.cn (W. Xu), yanhuili@nju.edu.cn (Y. Li), mingliangma@smail.nju.edu.cn (M. Ma), lchen@nju.edu.cn (L. Chen), zhouyuming@nju.edu.cn (Y. Zhou).

<https://doi.org/10.1016/j.scico.2026.103471>

Received 15 November 2025; Received in revised form 19 February 2026; Accepted 4 March 2026

Available online 8 March 2026

0167-6423/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

DL software follows the principles of a data-driven paradigm [10]: it takes a given DL program, encompassing model structure, loss function, and iteration settings, and learns the final DL model utilizing training data. This process leads to behavioral patterns in DL software that contrast with traditional software. Three key elements, i.e., DL program, DL model, and training data, shape the construction of DL software. Developers pay much attention to debugging techniques for DL software, primarily on the first two elements [11]. (a) DL program debugging aims to identify and remove program faults, as the DL program outlines the structure and training methodology of DL models. A novel instance of program debugging would be the learning-based framework, DeepFD [12], designed by Cao et al., which frames the fault localization task as a learning problem. (b) DL model debugging aims to uncover and pinpoint dubious regions within trained DL models, such as neurons or layers. To illustrate, Zhong et al. proposed Apricot [13], a weight-adaptation approach to debug DL models, which assumes that the weight values of neurons are one of the primary root causes of the misbehaviors of DL models.

Unlike traditional software, data is the foundational backbone for constructing DL software [14]. In contrast to models and programs, developers often pay less attention to data quality [15], frequently containing errors across various domains. In natural language processing datasets, label errors are widespread, undermining model fine-tuning and evaluation, ultimately degrading overall performance [16]. Likewise, the limited availability of rare case images undermines model accuracy in medical diagnosis, making it challenging to effectively identify and diagnose such cases in real-world applications [17]. Models trained on faulty datasets are error-prone, potentially leading to significant losses or severe consequences [17,18]. For instance, IBM's cancer diagnosis system suffered poor accuracy due to insufficient training data [19]. These challenges highlight the critical need for training data debugging to ensure the reliability of DL software. As a result, researchers have turned their attention to training data debugging, which is intended to identify and correct training data that causes errors in the DL models. For example, Li et al. highlighted a new debugging method, LTDD [20], designed to eliminate biased feature values in training data, thereby addressing unfairness issues. Meanwhile, Yin et al. introduced a dynamic data fault localization approach, DFauLo [18], to identify mislabeled and noisy data within deep learning datasets.

From the perspective of the DL software lifecycle [21], the process typically consists of two phases: development and iteration, where data errors may occur in both stages and compromise model quality. However, we recognize that current data debugging methodologies mainly focus on the development phase, overlooking the later iterations. Due to the inherent characteristics of DL software, its iterative process often involves model fine-tuning procedures [22]. We argue that data debugging techniques are even more necessary during the fine-tuning stage based on the following two points:

- During the fine-tuning stage, while the original model structure remains in use, the newly incorporated fine-tuning dataset plays a more crucial role in shaping the performance of the fine-tuned model.
- The fine-tuning datasets may be extracted from the application context with unavoidable noise or other interference factors (e.g., out-of-distribution data [23] and adversary samples [24]), which can introduce faults in the datasets.

Despite the critical importance of fine-tuning data, existing research lacks effective methods to detect and address dataset errors that arise specifically during the fine-tuning stage.

Building on the principles of software measurement [25], this paper aims to establish predictive models that identify and diagnose flaws in fine-tuning data by drawing software metrics from the fine-tuning procedures. At the heart of our approach is capturing the potential abnormal discrepancies between the pretraining and fine-tuning processes. We distinguish this disparity by comparing the difference in the metrics extracted during the two processes, referred to as change metrics.¹ Specifically, (a) we continue to use the runtime information (i.e., metrics) used in previous studies [12,27], extracting these metrics during the pretraining and fine-tuning stages; (b) we introduce two methods to calculate change metrics, computing the variation between descriptive statistics of the two processes, which yields the Descriptive Statistics Difference Metrics (DSDMs), and comparing disparity on multiple dimensions (distance, similarity, and distribution) to get the Multidimensional Data Comparison Metrics (MDCMs) (see details in Section 4.3); (c) we diagnose errors on fine-tuning data by building predictive models with five learners.

We evaluate our proposed change metrics on 233 DL models from six previously studied open datasets [12]. Initially, we create a benchmark fine-tuning dataset with distributions differing from the original dataset; this is achieved by adding noise to the original dataset. Using this benchmark dataset, we introduce errors by applying five mutation operators [28,29] to the data. Next, we pretrain the model using the original dataset and fine-tune the pretrained model using the benchmark dataset and mutated dataset. We gather runtime data for every epoch throughout pretraining and fine-tuning processes. From the data sequences of both processes, we calculate eight distinct descriptive statistics following previous studies [12,27]. The statistics from the fine-tuning process serve as the baseline metrics,² and the difference between the statistics of the two processes represents the DSDMs (one part of our change metrics). We also evaluate the disparities between the two, based on their data sequences, taking into account three dimensions: distance, similarity, and distribution disparity, giving us the MDCMs (the other part of our change metrics). We establish the presence of errors in the mutated dataset by noting the accuracy differences amongst the fine-tuning models. Finally, we use our change and baseline metrics to create predictive models with five learners. The experimental results indicate that (a) our change metrics are more effective than the baseline in fault diagnosis for fine-tuning data; (b) in most cases, using DSDM and MDCM together is more effective than using them separately; (c) the time cost of our approach primarily depends on the model training process, which is generally acceptable overall.

¹ The definition of 'change metrics' in our paper may be slightly different from the original definition [26] in traditional software: it is closer to the difference of metric values.

² These metrics have been proven to help identify errors in DL programs [12].

Our study makes the following contributions:

- **Dimension.** This study opens a new dimension of DL debugging, i.e., Fine-tuning Data Debugging, which focuses on identifying the data errors in the fine-tuning dataset.
- **Metrics.** This paper proposes DSDM and MDCM, two groups of change metrics extracted from the pretraining and fine-tuning stages in the lifecycle of DL software, which are effective indicators to present running differences between the pretraining and fine-tuning.
- **Study.** This paper contains an extensive empirical study with 233 DL models under five classifiers. The results of our experiment indicate that our change metrics are more effective than the baseline in fault diagnosis for fine-tuning data. To support reproducible study, we provide data and source code used to conduct this study at <https://github.com/ChangeMetric/CM>.

The rest of this paper is organized as follows: [Section 2](#) reviews the related works. [Section 3](#) presents the motivation and the main idea of our approach. [Section 4](#) provides a detailed introduction to our proposed approach. [Section 5](#) describes the experimental setup, and [Section 6](#) reports the experimental results. We discuss additional results and summarize the insights of our study in [Section 7](#). [Section 8](#) analyzes the threats to the validity of our study. Finally, we conclude our research work in [Section 9](#).

2. Related work

To deepen understanding of the existing research, we present three related lines of work in this paper:

Study of Bugs in DL Programs. With the rapid growth of deep learning (DL) applications across various domains, researchers have increasingly investigated bugs in DL systems. Researchers begin to study the bugs that may arise in deep learning applications. Zhang et al. [30] conducted an empirical study on TensorFlow program bugs and summarized four symptoms and seven root causes of the bugs. They also studied the challenges and the strategies for bug detection and localization. Islam et al. [31] studied more bugs about five popular deep learning libraries to understand the types of bugs, root causes of bugs, and impacts of bugs. Humbatova et al. [7] introduce a large taxonomy of faults in DL systems through manually analysing and structured interviews. Morovati et al. [32] compared the ML and non-ML bugs in ML-based software systems and found that fixing ML bugs is more costly and ML components are more error-prone. In addition to this, many researchers [33–35] have also studied bugs in DL frameworks such as TensorFlow.

Detection and Localization of DL Program Bugs. There are many research devoted to debugging DL programs. Wardat et al. [36] found that historic trends in values propagated between layers can be analyzed to identify and localize faults. Schoop et al. [37] proposed Umlaut, which checks DL program structure and model behavior against DL debugging heuristics and strategies used by experts. Nikanjam et al. [38] proposed NeuraLint, a model-based fault detection approach for DL programs, using meta-modeling and graph transformations. AutoTrainer [27] and DeepDiagnosis [39] detected and fixed common bugs in DL programs by monitoring the DNN training process. Cao et al. [12] and Wardat et al. [40] used the data-driven approach that leverages model features collected during the training process to learn problem patterns.

DL model testing. With the wide application of DL models, it is important to test them to ensure their quality. In recent years, researchers have tested DL models in many domains, such as autonomous driving [41–44], machine translation [45,46], etc. Pei et al. [47] proposed the first whitebox framework, DeepXplore, to test real-world DL systems and introduced neuron coverage as an evaluation criterion. Xie et al. [48] proposed DeepHunter, a coverage-guided fuzz testing framework with a metamorphic mutation strategy for detecting potential defects of general-purpose DNNs. Lee et al. [49] presented Adapt, a new white-box testing technique by effectively employing the neuron-selection strategy. Wang et al. [50] proposed a novel Black-box Efficient Testing (BET) method for CNN models with the insight that CNNs are generally prone to being affected by continuous perturbations. Yu et al. [51] put forward Test4Deep, an effective white-box testing DNN approach that is based on a differential testing framework to automatically verify inconsistent DNNs' behavior.

We emphasize that our work differs significantly from existing research in the following key respects: it lacks effective methods for detecting and addressing data errors during the fine-tuning stage. In contrast, we design and implement DSDM and MDCM—two sets of change metrics extracted from the pretraining and fine-tuning stages of the deep learning software lifecycle to build prediction models that detect data errors during the fine-tuning stage. These metrics serve as effective indicators of the operational differences between pretraining and fine-tuning.

3. Motivation

To illustrate the main idea of our method, we first introduce change metrics from traditional software engineering that inspired it. In software development and maintenance, code changes are necessary to remove inherent defects, enhance existing functions, refactor code, and improve performance. Metrics related to code changes are often extracted to detect bugs. Kim et al. [52] compared pre- and post-code-change values against traditional complexity metrics and derived new metrics by calculating differences. Šikić et al. [53] proposed aggregated change metrics, which are created by aggregating the data of all changes made to the software between two versions, taking into account the chronological order of the changes. Moser et al. [26] evaluated change and static code metrics using three standard machine learning classifiers on the Eclipse project and found that change metrics are more effective at predicting defects than code metrics. Choudhary et al. [54] investigated change metrics in conjunction with code metrics and found that change metrics positively impact the performance of fault prediction models.

As stated in [Section 1](#), similar to change metrics used in traditional software engineering, we aim to measure differences between the runtime data of two DL software lifecycle stages, i.e., pretraining and fine-tuning, to detect errors in fine-tuning data. Our main

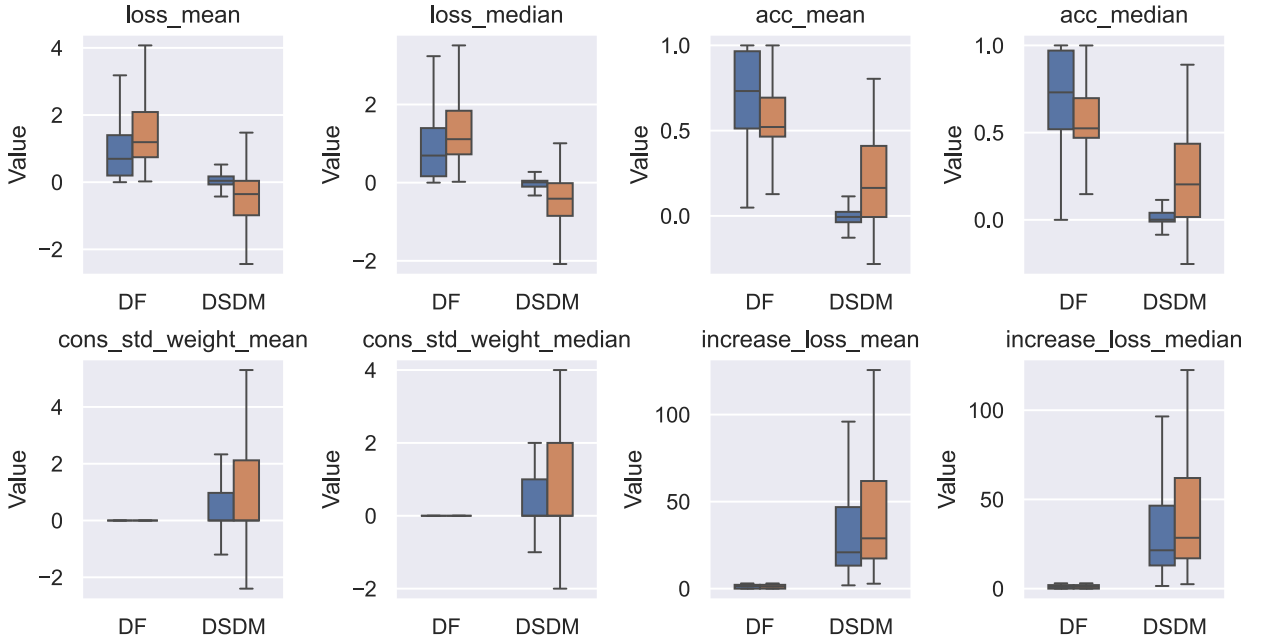


Fig. 1. Comparison of the original and change metric values on the non-faulty (DF) and faulty (DSDM) fine-tuning datasets.

idea is that fine-tuning can be viewed as a modification to the pretrained model, and because of this modification, the runtime data has also changed. We can determine whether the fine-tuned data contains errors by measuring changes (i.e., change metrics) in runtime data collected during pretraining and fine-tuning. *Intuitively, if the fine-tuning dataset has errors, some runtime data from the fine-tuning process should be significantly different from that of the pretraining process.* It is worth noting that our work is the first to apply this “change metrics” concept to data debugging.

We conduct a primitive investigation on runtime data to compare the differences between using existing runtime data directly and using change metrics. Fig. 1 shows the comparison under eight kinds of runtime data. DF represents the original diagnosis features of runtime data used in the previous study [12], and DSDM represents the differences (i.e., our change metrics) between pretraining and fine-tuning (see Sections 4.2 and 4.3 for more details).

We have the following two observations. (a) There are slight differences in the eight existing diagnosis features extracted from the fine-tuning stage between the faulty and non-faulty datasets, as shown in the ‘DF’ boxplots. In the four subplots above, the distribution ranges of the feature values are relatively close, resulting in substantial overlap. The four subplots below show minimal differences in the feature values, leading to almost complete overlap. (b) When computing the difference of the eight existing metrics extracted from the pretraining and fine-tuning stages, the differences become more significant between the faulty and non-faulty datasets, as shown in the ‘DSDM’ boxplots. The range of feature values shows a greater disparity, with a smaller overlap in their distributions.

These observations above indicate the potential of using change metrics to diagnose data bugs. In the next section, we will propose two groups of change metrics. Based on these metrics, we can train classification models to identify if there are errors in the fine-tuning dataset.

4. Proposed approach for fine-tuning data debugging

In this section, we present a detailed description of our approach.

We regard the problem of diagnosing dataset bugs as a classification problem and train a classification model to diagnose whether the fine-tuning dataset contains bugs and what kind of bugs it contains. Fig. 2 shows the outline of our approach. It comprises five steps: data mutation, model training, and runtime data collection, metrics calculation, labeling, and classification model training.

Firstly, we apply five different mutation operators to the original fine-tuning data to create faulty datasets. Next, we train the DL model with the pretraining data and then fine-tune it separately with the original fine-tuning data and mutated data. During the two training processes, we extract the runtime information, such as the loss, the accuracy, and the occurrences of NaN loss, at certain intervals (e.g., every epoch). Then, we calculate the difference between the descriptive statistics of the runtime information collected during the pretraining and fine-tuning processes. We also compare the disparity across multiple dimensions, such as distance and similarity. After that, we determine whether the model fine-tuned using the mutated dataset is buggy based on the model’s accuracy. Finally, we build the classification model using the previously collected metrics and labels.

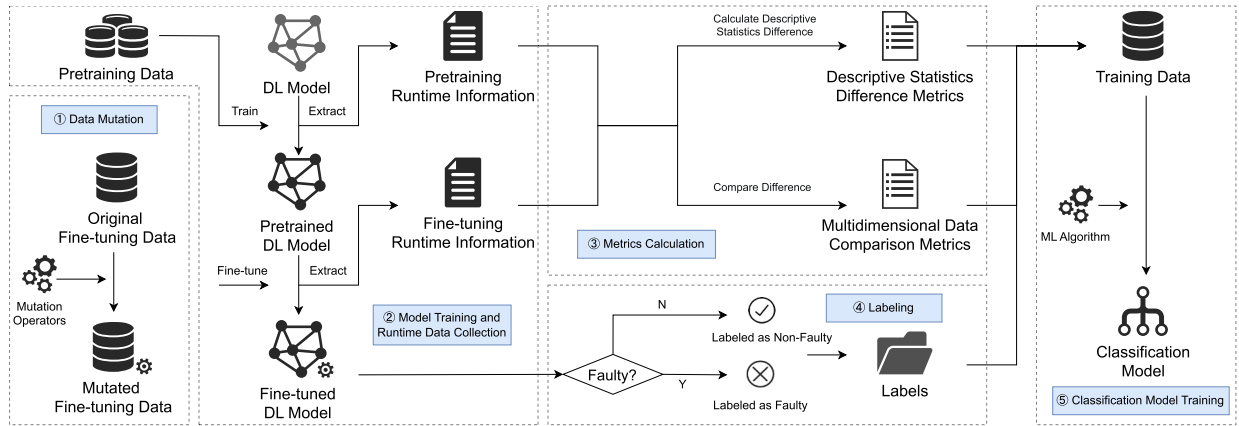


Fig. 2. Overview of our approach.

4.1. Data mutation module

Fine-tuning dataset plays a vital role in building DL models, as it can enhance the model's performance and generalization ability, enabling the model to adapt to specific tasks and domains. The data is usually extensive in size and labeled manually [55–57]. Collecting data is usually a laborious and sometimes error-prone process. Humans tend to make mistakes, and instruments tend to be inaccurate, so the collected data is inevitable to have some errors. The dataset contains irrelevant or meaningless data termed as noise, which can significantly affect various data analysis tasks of machine learning: classification, clustering, and association analysis [58]. We refer to these datasets that make the model less accurate as faulty datasets.

To build classification models for diagnosing data faults, we need enough training data. To this end, sufficient fine-tuning datasets that make the fine-tuned model less accurate are required. Note that the accuracy reduction here is not relative to the pretrained model but to the other fine-tuned models. More precisely, we would like to have a large number of fine-tuning datasets and be able to determine accurately whether these fine-tuning datasets have errors. However, collecting enough real-world dataset errors for training our classification model is challenging. Therefore, we use data mutation operators, designed based on observing potential problems that could occur during the data collection process and widely used in previous studies [28,29], to simulate errors in our dataset. Using mutation operators not only allows for the generation of a large volume of data but also facilitates the comparison between models trained on mutated datasets and those trained on original datasets, thereby determining whether the mutated datasets contain errors. Specifically, we use the following five mutation operators: noise perturbation, label error, data repetition, data missing, and data shuffle.

- **Noise Perturbation (NP).** The NP operator randomly adds noise to training data. A single data point might contain disturbances originating from diverse origins. For example, a camera-captured image could contain distortions induced by varying weather conditions (i.e., rain, snow, etc.). The NP operator tries to simulate potential issues relevant to noisy training data (e.g., NP introduces random alterations to certain pixels within an image).
- **Label Error (LE).** Each data point has two kinds of data: feature and label. Given that datasets are frequently extensive (e.g., the CIFAR-10 dataset [56] contains 50,000 training data), it is not unusual that some data points can be mislabeled. The LE operator injects such kind of faults by modifying the label associated with a particular data point.
- **Data Repetition (DR).** The DR operator duplicates a portion of the training data. The training data is often collected from multiple sources, some of which are quite similar, and the same data point can be collected more than once. Duplicate training data may lead to overfitting, causing the model to perform poorly on unseen data.
- **Data Missing (DM).** The DM operator removes some of the training data. This scenario may arise due to unintentional or mistaken deletion of certain data points. Missing data can lead to a shortage of data, thereby causing the model to underfit and be unable to adequately represent the features of the data.
- **Data Shuffle (DS).** The DS operator rearranges the order of the training data before the training process. In theory, running the training program with the same set of training data should yield the same DL model. However, in practice, the training procedure's implementation is often sensitive to the data's order. Developers may not prioritize the data's order when preparing training data, potentially overlooking these issues during training.

4.2. Model training and runtime data collection module

In this step, we first pretrain and fine-tune DL models and then perform a two-stage runtime data collection. The model training process contains a lot of information, such as the weights of the neurons and the loss of the model in each iteration. It's crucial to determine which information helps bug detection. There have been many recent studies [12,27,36,37,59] on dynamic trace collection

Table 1
Runtime data collected during DL model pretraining and fine-tuning process.

Runtime Data	Description
loss	Training set loss.
accuracy	Training set accuracy.
loss_val	Validation set loss.
acc_val	Validation set accuracy.
nan_loss	Occurrences of NaN (not a number) loss.
nan_accuracy	Whether NaN occurs in accuracy.
nan_weight	Whether NaN occurs in weight.
nan_gradient	Whether NaN occurs in gradient.
large_weight	Occurrences when the maximum weight exceeds a specific threshold.
decrease_acc	Occurrences when accuracy is lower than the previous recorded accuracy.
increase_loss	Occurrences when the loss is higher than the previous recorded loss.
cons_mean_weight	Occurrences when the weight mean is the same as the previous recorded value.
cons_std_weight	Occurrences when the weight standard deviation is consistent with the previous recorded value.
gap_train_test	Whether the difference between training accuracy and validation accuracy is excessively large.
test_turn_bad	Whether the training set loss is decreasing while the validation set loss is increasing.
slow_converge	Whether the improvement in accuracy of trained models occurs at a slow pace.
weight_change_little	Whether the weight changes little
dying_relu	In recent iterations, there's been a significant subset of neurons with zero gradients, which comprises a substantial portion of the entire DNN. Despite this, the neural network's accuracy remains low.
gradient_vanish	Whether there is a problem of gradient vanish.
gradient_explosion	Whether there is a problem of gradient explosion.

and runtime monitoring. We refer to the approach of their work. Specifically, we set callbacks to dynamically monitor the model's training status and capture critical information during pretraining and fine-tuning processes. Finally, we collect 20 kinds of runtime data, which is also used in [12], including many aspects, e.g., loss, accuracy, and the number of times they were NaN. We assume that there are no errors in the model structure and hyperparameters during model fine-tuning, and the only source of errors is the dataset.

Table 1 shows the specific runtime data and the detailed description. Note that some runtime data, such as nan_accuracy and nan_weight, have values in the range of True and False. For the convenience of subsequent calculation, we convert it to 1 and 0, as in the previous study [12]. As there are two training processes, i.e., pretraining and fine-tuning, we collect runtime data for these two processes separately. The runtime data are collected each epoch during the training process, resulting in 20 data sequences. We will compare these data sequences from pretraining and fine-tuning runtime data to calculate our change metrics.

4.3. Metrics collection module

As mentioned, we collect runtime data once per epoch during the model training process. Since the model undergoes multiple epochs during training, each set of runtime data forms a *data sequence*. We compare the differences between the data sequences of the pretraining process and the fine-tuning process from two aspects.

4.3.1. Descriptive statistics difference metric (DSDM)

We can use statistical methods to summarize each data sequence and then compare the differences between the summarized data. Followed by previous work [12], we use descriptive statistics to summarize the data. Specifically, we use 8 different statistical operators, as shown in Table 2.

These descriptive statistical operators include measures of data central tendency and dispersion. After applying the statistical operators, we can get 160 (20×8) diagnosis features (DFs)³ to characterize the training process of the given DL software. For DL software, pretraining and fine-tuning are two different processes, which means that the diagnostic features of the two are different. For each DF f , we take the difference between the DF values ($f_{pretrain}$ and f_{fine_tune}) extracted from pretraining and fine-tuning stages as our DSDM. It can be calculated as follows:

$$DSDM(f) = f_{pretrain} - f_{fine_tune}$$

Therefore, we get another 160 new features to characterize the difference between pretraining and fine-tuning processes.

4.3.2. Multidimensional data comparison metric (MDCM)

In addition to computing the difference of descriptive statistics, we can also directly compare the raw data sequences in multiple aspects. In this study, we compare the differences of each kind of data sequence collected during the pretraining and fine-tuning processes from three aspects using five metrics, as shown in Table 3.

Distance. To calculate the distance between two data sequences, we use Euclidean distance and Manhattan distance, which are commonly used in distance calculation. Given two data sequences $\mathbf{a} = (a_1, a_2, \dots, a_n)$ and $\mathbf{b} = (b_1, b_2, \dots, b_m)$, in which a_n and b_m

³ We use DFs to represent the 160 metrics summarized by 8 statistical operators, which are used in previous work [12].

Table 2
Descriptive statistical operators for runtime data aggregation.

Operators	Description
max	The maximum value in a data sequence.
min	The minimum value in a data sequence.
median	The median value of a data sequence.
mean	The mean value of a data sequence.
var	The variance of a data sequence.
std	The standard deviation of a data sequence.
skew	The skewness of a data sequence.
sem	The standard error of the mean of a data sequence.

Table 3
Multidimensional data comparison metrics used in this study.

Category	Metric	Description
Distance	ED	The Euclidean distance of the two data sequences.
	MD	The Manhattan distance of the two data sequences.
Similarity	CS	The cosine similarity of the two data sequences.
Distribution Difference	KST	The p-value obtained from conducting a Kolmogorov-Smirnov test on two data sequences.
	MMD	The maximum mean discrepancy of the two data sequences.

represent the runtime data of the n th and m th epoch. The number of epochs for pretraining and fine-tuning is usually not the same, and the distance metric requires both vectors to have the same dimensions. We need to cut out some of the more dimensional values for the calculation. Suppose $n \geq m$, we can calculate Euclidean and Manhattan distances as:

$$ED(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{k=1}^m (a_{n-m+k} - b_k)^2}$$

$$MD(\mathbf{a}, \mathbf{b}) = \sum_{k=1}^m |a_{n-m+k} - b_k|$$

Similarity. We use cosine similarity to evaluate the similarity between two data sequences. As cosine similarity also requires the data to have the same dimensions, we also cut out some of the more dimensional values. It can be calculated by the following formula:

$$CS(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_{k=1}^m a_{n-m+k} b_k}{\sqrt{\sum_{k=1}^m a_{n-m+k}^2} \cdot \sqrt{\sum_{k=1}^m b_k^2}}$$

Distribution Difference. To compare the distribution difference of two data sequences, we use the Kolmogorov-Smirnov test (KS test) [60] and maximum mean discrepancy (MMD) [61].

- KS test is a nonparametric test that can be used to test whether two samples came from the same distribution. It is a very efficient way to determine whether two samples are significantly different from each other. KS test doesn't require two samples to have the same dimensions, so we can perform the KS test directly on two data sequences, regardless of whether they have the same dimensions.
- MMD is a statistical metric used to measure the difference between two probability distributions. It quantifies the distance between them by comparing the means of sample sets from both distributions in the feature space. MMD is commonly used in kernel methods, where distributions' distances are computed by mapping samples to a high-dimensional feature space and then calculating the mean of kernel functions in that space. The MMD between two distributions P and Q can be calculated as:

$$MMD(P, Q) = \frac{1}{n(n-1)} \sum_{i=1}^n \sum_{j=1, j \neq i}^n k(x_i, x_j) - \frac{2}{nm} \sum_{i=1}^n \sum_{j=1}^m k(x_i, y_j) + \frac{1}{m(m-1)} \sum_{i=1}^m \sum_{j=1, j \neq i}^m k(y_i, y_j)$$

where x_i and x_j represent samples from distribution P , and y_i and y_j represent samples from distribution Q . $k(x_i, x_j)$, $k(x_i, y_j)$ and $k(y_i, y_j)$ represent the kernel functions applied to pairs of samples. n and m are the number of samples from distribution P and Q .

In a word, we calculate five comparison metrics for 20 kinds of runtime data. Thus 100 (20×5) new features can be obtained.

4.4. Labeling module

In previous steps, we generated numerous mutated datasets and used them to fine-tune the DL model. However, not all the mutated datasets are necessarily faulty (i.e., cause the accuracy of the DL model to decrease). In some cases, the model fine-tuned with the

mutated dataset achieves a higher accuracy than the one fine-tuned with the original dataset. Therefore, it's essential for us to assess the labels of each mutated dataset. Therefore, instead of assuming that all mutations result in data errors, we aim to identify mutated datasets that are likely to introduce harmful data issues. Due to the randomness of DL model training, slight variations in the DL model's accuracy are natural. For each fine-tuning process, we repeat it 5 times to mitigate the effect of randomness. As a result, we get a sequence of accuracy values.

To assess whether a mutated dataset is likely to be faulty, we compare the accuracy sequence of the model fine-tuned on the mutated dataset with that of the model fine-tuned on the original dataset, a practical criterion that has been adopted in a prior study [12]. Based on previous works [12,62,63], we employ the following two-step procedure to determine whether the mutated dataset contains errors.

- First, we use the generalized linear model (GLM) [64] and Cohen's d [65] to quantify the statistical significance and the effect size. If the results are significantly different (i.e., the p -value is less than 0.05 and the effect size is not less than 0.2).
- We then check whether the mean accuracy of the DL model fine-tuned with the mutated dataset is lower than that with the original dataset.

If the above two requirements are satisfied simultaneously, the mutated dataset is considered faulty. Otherwise, we label the dataset as non-faulty.

4.5. Classification model training module

We treat data fault diagnosis as a classification problem. Therefore, we select five widely used machine learning algorithms for classification tasks [66–68]: Decision Tree (DT), Random Forest (RF), Naive Bayes (NB), K-Nearest Neighbors (KNN), and Logistic Regression (LR).

- The DT predicts the value of a target variable by learning simple decision rules inferred from the data features [69].
- The RF is an ensemble learning method, which operates by constructing a multitude of decision trees at training time, and the output of the random forest is the class selected by most trees [70].
- The NB classifiers are a family of simple “probabilistic classifiers” based on applying Bayes’ theorem with strong (naive) independence assumptions between the features [71].
- The KNN classifies an object by a plurality vote of its neighbors, with the object being assigned to the class most common among its k nearest neighbors [72].
- The LR describes the probabilities of the possible outcomes of a single trial are modeled using a logistic function [73].

5. Experiment setup

In this section, we will introduce the experiment setup.

5.1. Research question

We organize our experiments by addressing the following three research questions (RQs):

- RQ1: How effective are change metrics in fault diagnosis compared with baseline metrics?
- RQ2: How important are DSDM and MDCM in fault diagnosis?
- RQ3: How efficient is our approach?

5.2. Studied datasets and models

We select six popular datasets to conduct our experiment: Circle [74], Blob [75], MNIST [76], CIFAR-10 [56], IMDB [77], and Reuters [78]. Table 4 shows the detailed description. Circle and Blob are two datasets from Sklearn [79] for classification tasks, both of which have 700 training samples and 300 testing samples. MNIST is a gray-scale image dataset used for handwritten digit recognition with 60,000 training samples and 10,000 testing samples. Samples in MNIST are 28×28 pixel gray-scale images to denote handwritten digits from 0 to 9. CIFAR-10 is a colored image dataset used for object recognition. It contains 60,000 32×32 pixel color images (50,000 for training and 10,000 for testing), which are equally distributed into 10 classes, e.g., cat, dog, ship, and truck. IMDB is a movie review dataset for sentiment analysis with 25,000 training samples and 25,000 testing samples labeled by sentiment (positive/negative). Reuters is a newswire dataset for document classification with 11,228 samples labeled over 46 topics. The Reuters dataset exhibits significant class imbalance, with the top two categories alone accounting for over 50% of the data, despite having 46 categories in total. To ensure that the quality of the model is influenced only by the introduced mutation operators, we need to eliminate other sources of interference, including class imbalances. Therefore, for the Reuters dataset, we opted for the top two categories with the highest frequency and balanced the number of instances between these two categories to create the final dataset. As each data sequence in Reuters and IMDB has different dimensions, we fix the dimension to 300, as in previous work [12]. Sequences shorter than 300 are padded with 0 until the length is 300. Sequences longer than 300 are truncated so that they fit the desired length.

For the DL models, we use the datasets studied in recent works [12,27]. Specifically, the work [27] provided 233 normal DL models, which have been confirmed by model authors and their training scripts of various DL model structures (convolutional neural network, recurrent neural network, and fully connected layers only) for these six datasets.

Table 4
Studied Datasets and DL Models.

	Training set	Testing set	Dimension	Class	Model
Blob	700	300	2	3	39
Circle	700	300	2	2	36
MNIST	60,000	10,000	28×28	10	78
CIFAR-10	50,000	10,000	$32 \times 32 \times 3$	10	35
Reuters*	3876	970	300	2	32
IMDb	25,000	25,000	300	2	13

*To mitigate the impact of data imbalance, we selected the two categories with the largest number in the Reuters dataset.

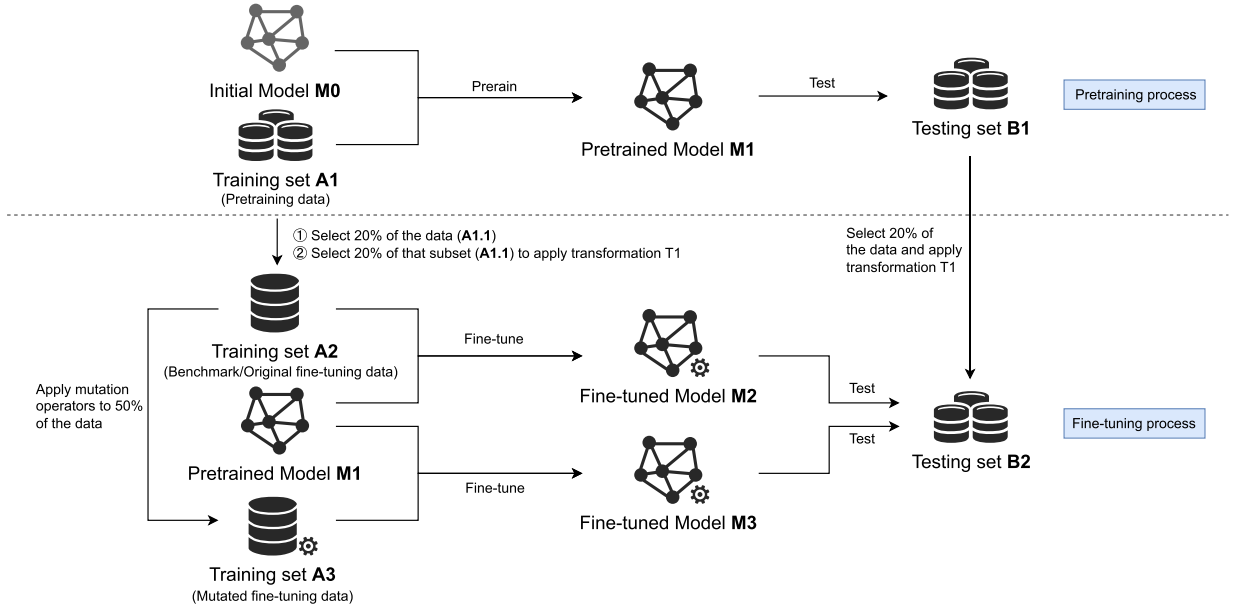


Fig. 3. The dataset construction and the detailed training process of our experiment.

5.3. Experiment settings

Our experiments are based on the fine-tuning scenario. This subsection elaborates further on the construction of the datasets used in the experiments and the related settings.

5.3.1. Fine-tuning dataset construction

In typical fine-tuning scenarios, the distribution of the fine-tuning dataset often differs from that of the pretraining dataset, and the model needs to adjust its parameters to adapt to the new application domain. However, it is challenging to obtain real-world fine-tuning datasets with verified errors. To validate the effectiveness of our method in diagnosing errors in the fine-tuning dataset, we use the simulated datasets.

We assume that the pretraining dataset (i.e., the six datasets introduced in Section 5.2) contains no errors. Based on this assumption, we construct the fine-tuning datasets. The training set A1 in Fig. 3 is used in pretraining, and it is taken from the original training set of the studied dataset described in Section 5.2. The testing set B1 is the original testing set of the studied dataset. We select 20% of B1 and apply a specific transformation T1 to construct a dataset with a different distribution, which results in testing set B2. In this study, we use the addition of Gaussian noise to the dataset as the transformation T1. Gaussian noise is a kind of random noise with a probability density function equal to that of the normal distribution, which is widespread in the real world. We select 20% of A1 to fine-tune the pretrained model. To ensure that the training set of the fine-tuning has a similar distribution to the test set, we select 20% of the data and apply the same transformation T1, which results in A2. In this way, the model learns under pretraining to handle a clean, noise-free classification scenario, whereas fine-tuning adjusts the model to cope with the noisy input distribution.

We take A2 as the *benchmark* fine-tuning training set (original fine-tuning data in Fig. 2), representing a scenario where fine-tuning on it does not introduce errors. To simulate faulty fine-tuning datasets, we select 50% of A2 to apply the mutation operator described in Section 4.1, resulting in the mutants A3 (mutated fine-tuning data in Fig. 2). Note that for the DS operator, we apply it to 100% of A2 as it has less impact on the model. To ensure class balance, we employ stratified sampling in all the aforementioned samplings.

Table 5
The statistics of the generated faulty mutants.

Dataset	Model	Faulty Mutant	Types of Faults				
			Noise Perturbation	Label Error	Data Repetition	Data Missing	Data Shuffle
Blob	39	390	55	120	85	70	60
Circle	36	455	120	135	50	110	40
MNIST	78	805	245	380	35	110	35
CIFAR-10	35	280	90	155	0	35	0
Reuters	32	210	65	100	15	30	0
IMDb	13	70	10	45	5	5	5
Total	233	2210	585	935	190	360	140

We use the method described in Section 4.4 to determine the label of the mutated fine-tuning data. Table 5 shows the statistics of the generated faulty mutants.

5.3.2. DL model pretraining and fine-tuning

As shown in Fig. 3, we use the initial model M0 and the Training set A1 to train the pretrained model M1 with the provided hyperparameters. We fine-tune the pretrained model M1 using A2 and A3, respectively, to obtain M2 and M3. During the fine-tuning process, all hyperparameters remain the same as during pretraining, except for the epoch, which is set to 5 as in the work of Shen et al. [24] and Hu et al. [22]. To mitigate the randomness, we repeat the fine-tuning process 5 times. Furthermore, during all training processes, we divide 25% of the training set into the validation set.

5.3.3. Baseline

We use the Diagnosis Features (DFs) as the baseline, which is calculated by applying 8 statistical operators to the 20 kinds of runtime data. The baseline contains 160 metrics and is used in previous work [12]. It achieves good performance on the fault diagnosis of DL programs with three classification algorithms.

5.4. Evaluation indicators

For the classification model, we introduce three indicators to evaluate the model performance.

Accuracy. It shows how often a classification ML model is correct overall, which can be calculated as:

$$Accuracy = \frac{TP + TN}{Total}$$

where TP denotes the true positive, TN denotes the true negative, and *Total* denotes all samples.

F1-score. It is calculated from the precision and recall of the test, where the precision is the number of true positive results divided by the number of all positive results, including those not identified correctly, and the recall is the number of true positive results divided by the number of all samples that should have been identified as positive. The F1-score is the harmonic mean of the precision and recall, which can be calculated as:

$$F1 - score = \frac{2TP}{2TP + FP + FN}$$

where FP denotes the false positive, and FN denotes the false negative. As our experiments are multiclass classification tasks, we use Macro-F1, which is calculated using the arithmetic mean of all the per-class F1-scores instead.

AUC. AUC represents the probability that the classifier ranks a positive instance higher than a negative instance. It stands for the area under the ROC Curve (receiver operating characteristic curve), which is a graph showing the performance of a classification model at all classification thresholds. In this article, we will calculate the AUC for each class separately and then take their average as the final result.

5.5. Analysis method

To quantify the improvement of performance, we first use the **Wilcoxon signed-rank test** [80] to check whether the difference between our model and baseline model is statistically significant at the significance level of 0.05. S, we introduce **Cliff's delta** δ [81] to measure the effect size. The effect size of Cliff's delta δ is usually assessed by the thresholds: $|\delta| < 0.147$ as negligible, $0.147 \leq |\delta| < 0.330$ as small, $0.330 \leq |\delta| < 0.474$ as medium, and $|\delta| \geq 0.474$ as large. Third, we introduce “**W/T/L**” [8,82,83] to denote one model performing better than, equal to, or worse than another model. Specifically, for model *A* and model *B*, we mark *A* as a “Win”, if (1) *A* performs significantly better than *B* according to the Wilcoxon signed-rank test (i.e. *p*-value is less than 0.05) and (2) the magnitude of the difference between *A* and *B* is not trivial according to Cliff's delta δ ($\delta \geq 0.147$). In contrast, we mark *A* as a “Lose”, if (1) *B* performs significantly better than *A* according to the Wilcoxon signed-rank test and (2) the magnitude of the difference between *A* and *B* is not trivial according to Cliff's delta δ ($\delta \leq -0.147$). Otherwise, we mark *A* as a “Tie”.

Table 6
Comparison of DSDM + MDCM and DF on the diagnosis of faults between five classifiers (RQ1).

Indicator	Dataset	DT		RF		NB		KNN		LR		W/T/L
		DSDM+MDCM	DF	DSDM+MDCM	DF	DSDM+MDCM	DF	DSDM+MDCM	DF	DSDM+MDCM	DF	
Accuracy	Blob	0.948	0.917	0.969	0.955	0.272	0.166	0.800	0.782	0.789	0.738	27/4/4
	Circle	0.934	0.887	0.971	0.925	0.169	0.333	0.735	0.686	0.770	0.666	
	MNIST	0.955	0.928	0.967	0.945	0.090	0.220	0.790	0.832	0.805	0.812	
	CIFAR-10	0.935	0.912	0.952	0.946	0.558	0.446	0.760	0.750	0.836	0.735	
	Reuters	0.944	0.914	0.960	0.933	0.281	0.201	0.822	0.811	0.910	0.841	
	IMDb	0.964	0.942	0.985	0.967	0.299	0.478	0.831	0.838	0.928	0.857	
	All	0.934	0.899	0.965	0.938	0.142	0.096	0.785	0.772	0.757	0.716	
Macro-F1	Blob	0.917	0.867	0.949	0.926	0.292	0.194	0.636	0.619	0.507	0.360	28/4/3
	Circle	0.909	0.835	0.957	0.837	0.179	0.337	0.568	0.491	0.632	0.325	
	MNIST	0.852	0.784	0.865	0.775	0.125	0.257	0.462	0.429	0.379	0.379	
	CIFAR-10	0.897	0.837	0.918	0.912	0.462	0.379	0.393	0.390	0.542	0.340	
	Reuters	0.776	0.732	0.810	0.702	0.292	0.181	0.346	0.321	0.558	0.367	
	IMDb	0.860	0.646	0.782	0.581	0.319	0.430	0.318	0.254	0.424	0.340	
	All	0.865	0.801	0.922	0.860	0.131	0.101	0.539	0.493	0.336	0.248	
AUC	Blob	0.957	0.920	0.956	0.936	0.664	0.567	0.769	0.746	0.684	0.602	28/5/2
	Circle	0.944	0.904	0.965	0.890	0.593	0.723	0.732	0.674	0.745	0.596	
	MNIST	0.916	0.882	0.899	0.845	0.585	0.659	0.684	0.678	0.665	0.658	
	CIFAR-10	0.928	0.890	0.924	0.922	0.781	0.727	0.615	0.589	0.718	0.554	
	Reuters	0.868	0.843	0.875	0.798	0.752	0.599	0.594	0.575	0.734	0.604	
	IMDb	0.945	0.824	0.880	0.781	0.814	0.836	0.621	0.572	0.700	0.629	
	All	0.922	0.883	0.932	0.883	0.570	0.547	0.707	0.670	0.612	0.556	

1. The numbers in the table represent the scores of the indicators, and higher scores are shown in **bold**.

2. The gray background indicates that the metrics DSDM+MDCM win over DF, i.e., the p -value is less than 0.05, and the δ is greater than 0.147.

6. Experiment results

In this section, we present the results of our experiments and answer the three RQs.

RQ1. Effectiveness of Fault Diagnosis.

Motivation and Approach. In this research question, we aim to assess the efficacy of the classification model trained using our proposed metric versus the classification model trained using baseline metrics for data fault diagnosis. We utilize the procedure depicted in Fig. 3 to pretrain and fine-tune the DL models, collect runtime information, and calculate our proposed metrics according to the methods described in Sections 4.2 and 4.3. For models trained using the benchmark fine-tuning training set, we treat them as error-free. Then, we calculate the labels for other models using the method described in Section 4.4. Finally, we train and test the classification model with five different classification algorithms described in Section 4.5 using the above data. The training and testing sets are split in a proportion of 7:3. To mitigate randomness, for each classification algorithm, we randomly partition the dataset and train the classification model 10 times, taking the average of the 10 results as the final outcome. For the hypothesis testing mentioned in Section 5.5, we use the results of 10 replications for comparison.

Results. Table 6 presents the results of the classification models trained using the data obtained from the models corresponding to the six datasets alone and the results of the classification model trained using all the data. The numbers in the table represent the average of 10 training results. We use bold to indicate the higher score of the two. We can observe that DSDM + MDCM performs best in most cases when using the RF classifier, while the performance is worst when using the NB classifier. The highest achieved values for Accuracy, Macro-F1, and AUC are 0.985, 0.957, and 0.965, respectively. When it comes to the comparison of our proposed metrics and baseline metrics, for all three indicators, the number of times the mean values of DSDM + MDCM are higher than DF is 29, 32, and 32, respectively. This indicates that our proposed metrics perform better than the baseline metrics in most cases. The gray background further presents the results of the hypothesis testing. It stands for DSDM + MDCM wins DF, i.e., the Wilcoxon p -value is less than 0.05, and the Cliff's delta is greater than 0.147. The last column shows the W/T/L results. For each indicator, the times DSDM + MDCM wins DF are 27, 28, and 28, accounting for 77%, 80%, and 80%, respectively. These results show that our proposed metrics significantly outperform baseline metrics in most cases.

Answer to RQ1: Our proposed metrics are most effective using the RF classifier. Furthermore, our proposed metrics can achieve significantly better accuracy, macro-F1, and AUC than DF on the five classifiers and seven datasets.

RQ2. Importance of DSDM and MDCM in Fault Diagnosis

Motivation and Approach. Through RQ1, we observe that our proposed metrics perform significantly better than the baseline metrics on data fault diagnosis. However, our proposed metrics contain two parts: DSDMs and MDCMs. We wonder how much each part affects the performance of the classification model. Therefore, we conduct an ablation experiment to study the effect of the two parts on this research question. To this end, we separately exclude MDCMs and DSDMs while training the classification model and compare their results with those without exclusion. Other settings remain the same as RQ1.

Table 7
Comparison of DSDM and DSDM + MDCM on the diagnosis of faults between five classifiers (RQ2).

Indicator	Dataset	DT		RF		NB		KNN		LR		W/T/L
		DSDM	DSDM+MDCM	DSDM	DSDM+MDCM	DSDM	DSDM+MDCM	DSDM	DSDM+MDCM	DSDM	DSDM+MDCM	
Accuracy	Blob	0.948	0.948	0.969	0.969	0.261	0.272	0.793	0.800	0.760	0.789	2/15/18
	Circle	0.935	0.934	0.977	0.971	0.120	0.169	0.689	0.735	0.703	0.770	
	MNIST	0.944	0.955	0.965	0.967	0.050	0.090	0.788	0.790	0.782	0.805	
	CIFAR-10	0.931	0.935	0.944	0.952	0.533	0.558	0.750	0.760	0.812	0.836	
	Reuters	0.946	0.944	0.964	0.960	0.238	0.281	0.795	0.822	0.876	0.910	
	IMDb	0.962	0.964	0.971	0.985	0.295	0.299	0.851	0.831	0.915	0.928	
	All	0.937	0.934	0.963	0.965	0.130	0.142	0.770	0.785	0.740	0.757	
Macro-F1	Blob	0.915	0.917	0.949	0.949	0.270	0.292	0.616	0.636	0.422	0.507	0/18/17
	Circle	0.907	0.909	0.964	0.957	0.117	0.179	0.509	0.568	0.483	0.632	
	MNIST	0.843	0.852	0.856	0.865	0.065	0.125	0.451	0.462	0.341	0.379	
	CIFAR-10	0.884	0.897	0.911	0.918	0.452	0.462	0.357	0.393	0.507	0.542	
	Reuters	0.795	0.776	0.834	0.810	0.253	0.292	0.361	0.346	0.458	0.558	
	IMDb	0.769	0.860	0.668	0.782	0.283	0.319	0.306	0.318	0.388	0.424	
	All	0.872	0.865	0.917	0.922	0.110	0.131	0.513	0.539	0.285	0.336	
AUC	Blob	0.948	0.957	0.957	0.956	0.654	0.664	0.757	0.769	0.640	0.684	0/17/18
	Circle	0.947	0.944	0.968	0.965	0.553	0.593	0.687	0.732	0.666	0.745	
	MNIST	0.911	0.916	0.893	0.899	0.529	0.555	0.674	0.684	0.631	0.665	
	CIFAR-10	0.919	0.928	0.918	0.924	0.782	0.781	0.572	0.615	0.690	0.718	
	Reuters	0.878	0.868	0.882	0.875	0.709	0.752	0.590	0.594	0.665	0.734	
	IMDb	0.882	0.945	0.818	0.880	0.787	0.814	0.604	0.621	0.679	0.700	
	All	0.923	0.922	0.927	0.932	0.551	0.570	0.686	0.707	0.586	0.612	

1. The numbers in the table represent the scores of the indicators, and higher scores are shown in **bold**.

2. The gray background indicates that the metrics DSDM lose to DSDM+MDCM, i.e., the p -value is less than 0.05, and the δ is less than -0.147.

Table 8
Comparison of MDCM and DSDM + MDCM on the diagnosis of faults between five classifiers (RQ2).

Indicator	Dataset	DT		RF		NB		KNN		LR		W/T/L
		MDCM	DSDM+MDCM	MDCM	DSDM+MDCM	MDCM	DSDM+MDCM	MDCM	DSDM+MDCM	MDCM	DSDM+MDCM	
Accuracy	Blob	0.948	0.948	0.967	0.969	0.295	0.272	0.822	0.800	0.738	0.789	12/10/13
	Circle	0.930	0.934	0.967	0.971	0.294	0.169	0.812	0.735	0.744	0.770	
	MNIST	0.941	0.955	0.961	0.967	0.280	0.090	0.803	0.790	0.836	0.805	
	CIFAR-10	0.924	0.935	0.946	0.952	0.477	0.558	0.778	0.760	0.810	0.836	
	Reuters	0.940	0.944	0.950	0.960	0.254	0.281	0.834	0.822	0.893	0.910	
	IMDb	0.976	0.964	0.979	0.985	0.603	0.299	0.818	0.831	0.930	0.928	
	All	0.925	0.934	0.958	0.965	0.182	0.142	0.813	0.785	0.747	0.757	
Macro-F1	Blob	0.909	0.917	0.943	0.949	0.336	0.292	0.696	0.636	0.331	0.507	11/10/14
	Circle	0.903	0.909	0.949	0.957	0.333	0.179	0.706	0.568	0.517	0.632	
	MNIST	0.810	0.852	0.846	0.865	0.298	0.125	0.408	0.462	0.427	0.379	
	CIFAR-10	0.858	0.897	0.892	0.918	0.406	0.462	0.432	0.393	0.463	0.542	
	Reuters	0.826	0.776	0.796	0.810	0.296	0.292	0.380	0.346	0.480	0.558	
	IMDb	0.790	0.860	0.742	0.782	0.554	0.319	0.296	0.318	0.377	0.424	
	All	0.849	0.865	0.906	0.922	0.199	0.131	0.607	0.539	0.301	0.336	
AUC	Blob	0.944	0.957	0.953	0.956	0.710	0.664	0.810	0.769	0.602	0.684	11/8/16
	Circle	0.945	0.944	0.958	0.965	0.706	0.593	0.807	0.732	0.694	0.745	
	MNIST	0.896	0.916	0.886	0.899	0.700	0.555	0.666	0.684	0.696	0.665	
	CIFAR-10	0.908	0.928	0.907	0.924	0.750	0.781	0.645	0.615	0.669	0.718	
	Reuters	0.903	0.868	0.861	0.875	0.723	0.752	0.609	0.594	0.687	0.734	
	IMDb	0.881	0.945	0.857	0.880	0.898	0.814	0.600	0.621	0.669	0.700	
	All	0.913	0.922	0.918	0.932	0.613	0.570	0.736	0.707	0.599	0.612	

1. The numbers in the table represent the scores of the indicators, and higher scores are shown in **bold**.

2. The gray background indicates that the metrics MDCM lose to DSDM+MDCM, i.e., the p -value is less than 0.05, and the δ is less than -0.147.

Results. Table 7 shows the comparison of the performance of classification models when using only DSDM and DSDM + MDCM for training. We find that, for three indicators, the scores of DSDM are lower than DSDM + MDCM on NB, KNN, and LR classifiers in most cases, while for DT and RF classifiers, there are some cases where the DSDM scores are higher, and in other cases, the DSDM + MDCM scores are higher. Overall, DSDM + MDCM scores higher on all three indicators in more cases, specifically 29, 30, and 28 times, respectively. For the hypothesis testing results shown in the last column, we find that using DSDM only wins 2 times in terms of Accuracy. They lose 18, 17, and 18 times in terms of Accuracy, Macro-F1, and AUC, respectively. This indicates that MDCM plays an important role.

Table 8 presents the comparison of the performance of classification models when using only MDCM and hybrid metrics (DSDM + MDCM) for training. We observe that in most cases, the scores of MDCM are lower than DSDM + MDCM on DT, RF, and LR classifiers, while they are higher on NB and KNN classifiers. Overall, the hybrid metrics (DSDM + MDCM) score higher than MDCM 21, 22, and 22 times, respectively. For the hypothesis testing results, MDCM loses 13, 14, and 16 times under the three indicators and wins 12, 11, and 11 times. While the numbers of losses and wins are relatively close, there are slightly more losses. This indicates that in some cases, the performance of the classification model improves after removing DSDM, while in others, it deteriorates.

Table 9

The average time (s: second) taken by our approach (RQ3).

Dataset	Runtime data collection		Change metrics calculation (s)	Classification model training (s)
	Pretrain (s)	Fine-tune (s)		
Blob	4.53	1.00	0.07	0.54
Circle	5.94	0.96	0.07	0.50
MNIST	373.97	5.96	0.05	1.02
CIFAR-10	377.88	6.10	0.09	0.28
Reuters	3806.96	172.14	0.05	0.33
IMDb	16585.34	763.02	0.06	0.14

From the above analysis, we can conclude that both DSDM and MDCM play a certain role in constructing classification models, with MDCM having a relatively larger impact compared to DSDM. However, when used in conjunction, the performance of the classification models is better than when used individually.

Answer to RQ2: In fault diagnosis, the importance of both MDCM and DSDM cannot be ignored, but relatively speaking, the role of MDCM is slightly greater than that of DSDM.

RQ3. Efficiency of Our Approach

Motivation and Approach. In this research question, we evaluate the efficiency of our approach. We divide the time consumption into runtime data collection, change metrics calculation, and classification model training. Runtime data collection occurs during the model training process, so we record the total training time of the model. We separately calculate the average time consumption of each model for the pretraining and fine-tuning stages, categorized by dataset. For the other two parts, we calculate the average time taken for metric calculation for each fine-tuned model and the average time taken for training each classification model.

Results. Table 9 presents the results. We find that our approach spends the most time on runtime data collection, and the time differences vary significantly for different datasets, ranging from a few seconds to several tens of thousands of seconds. This is because runtime data collection is performed during training, and training deep learning models is highly time-consuming. The differences in time are due to variations in the sizes of models corresponding to different datasets. Metric calculation and classification model training can generally be completed within a second. Overall, the time overhead of our method is acceptable.

Answer to RQ3: Our approach spends a considerable amount of time on runtime data collection, with the time taken depending on the size of the model. However, metrics calculation and classification model training can generally be completed within a second, making the overall time consumption acceptable.

7. Discussion

In this section, we discuss other experiment results and findings of our study.

7.1. Classification power of individual metrics

In RQ2, we study the importance of DSDM and MDCM in fault diagnosis through ablation experiments. In this discussion, we aim to explore to what extent each metric contributes to fault diagnosis. To this end, we calculate the information gain ratio [84–86] of the 420 metrics (160 DFs, 160 DSDMs, and 100 MDCMs) using all six datasets. Metrics with higher information gain ratio values are usually considered more powerful in classification. Therefore, we sort the metrics in descending order of gain ratio and select the top 50 metrics.

Fig. 4 shows the results. The x -axis represents each metric, with our proposed metrics marked in red. With runtime data A (see details in Table 1), descriptive statistical operator B (see details in Table 2), and multidimensional data comparison metric C (see details in Table 3), we name the corresponding DF as A_B , the corresponding DSDM as A_B_DSDM , and the corresponding MDCM as A_C . For example, decrease_acc_mean represents the mean value of the runtime data decrease_acc. Among the top 50 metrics ranked by information gain ratio values, 37 are our proposed metrics, accounting for 74%. Besides, our proposed metrics include the top 10 metrics with the highest information gain ratio. Therefore, it is evident that our change metrics have stronger classification power.

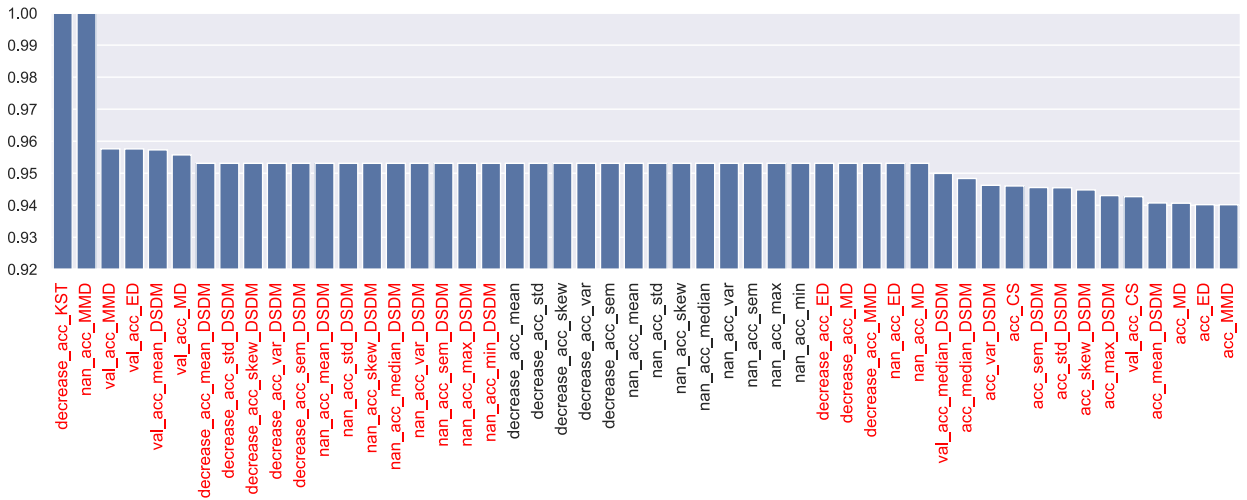


Fig. 4. Top-50 metrics ranked by the normalized information gain ratio values.

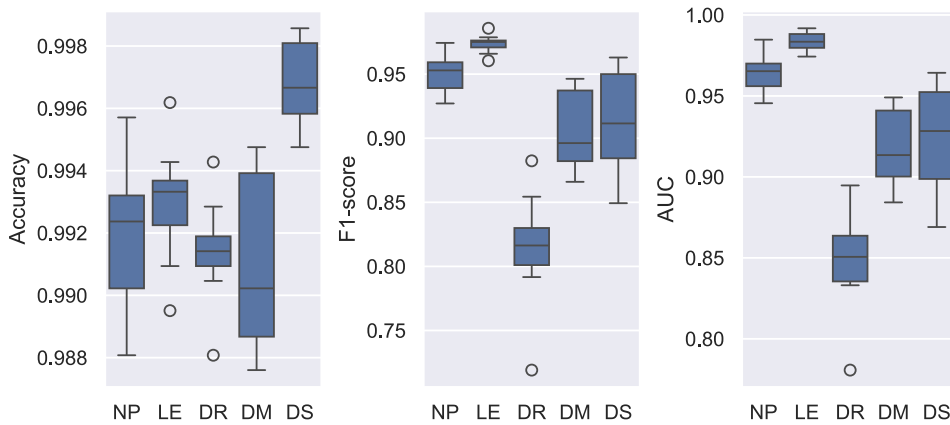


Fig. 5. Effectiveness of the RF classifier trained using all datasets on diagnosing different fault types.

7.2. Effectiveness of fault diagnosis for different classes

In RQ1, we evaluate the overall performance of the change metrics on fault diagnosis. However, we note variations in the classification models' efficacy across different fault types. Therefore, in this discussion, we calculate the accuracy, F1-score, and AUC of each fault type of classification model to explore its effectiveness across diverse fault types.

Fig. 5 presents the results of the RF classifier. Each box plot represents the results of an indicator, with the x-axis representing error categories and the y-axis representing the indicators' score. We find that, in terms of F1-score and AUC, the classification models perform better on faults categorized as NP and LE compared to the other three types of faults. However, concerning Accuracy, the classification models demonstrate the best performance in classifying DS faults. This result aligns with our intuition. NP and LE alter the data, significantly impacting data quality. This makes it easier for the classifier to distinguish these two types of faults from the other categories. The other three operators merely change the quantity or order of data without modifying the data itself, so its effect on quality is minimal, which makes it harder to classify.

7.3. Principal component analysis on change metrics

Our change metrics consist of two parts: DSDMs and MDCMs, with 160 and 100 metrics in each part, respectively, making a total of 260 metrics. With so many metrics, there may be some redundant information, increasing computational costs. Therefore, we aim to explore the possibility of improving the model's performance by attempting dimensionality reduction through Principal Component Analysis (PCA) [87]. Specifically, we use the *sklearn* package in Python to perform PCA and select the top n principal components that cumulatively contribute to over 99.99% of the variance. Then, we train the classification model using the seven datasets, i.e., the six different datasets and the overall dataset obtained by merging the six individual datasets, after PCA processing

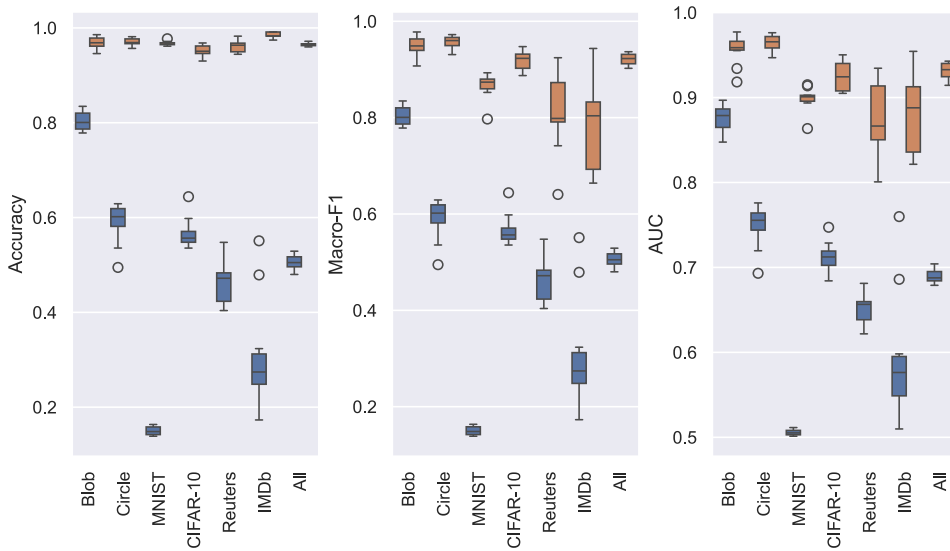


Fig. 6. Comparison of the performance of RF classifier with (■) and without (■) PCA process.

and calculating the Accuracy, Macro-F1, and AUC. Due to the superior performance of the RF classifier, as observed in the previous research questions, we conduct experiments using the RF classifier in this discussion.

Fig. 6 compares the results with and without PCA. We observe that after performing PCA, change metrics are less effective at diagnosing faults for all seven datasets. The values of all the three indicators show varying degrees of decrease. The classification model trained using metrics extracted from the MNIST dataset shows the most significant performance degradation. Therefore, we guess that since PCA does not consider label information, it may have resulted in the loss of some crucial information for classification, thus leading to a decrease in performance.

7.4. The impact of different mutation proportion

In Section 5.3, mutation operators are applied to 50% of the dataset A2 (with the DS operator applied to 100%) to create the mutated fine-tuning dataset. However, different mutation proportions may have varying impacts on the dataset's quality. This discussion investigates the effects of different mutation proportions for the five mutation operators used in this paper.

Specifically, for the NP, LE, DR, and DM operators, we experiment with mutation proportions of 10%, 20%, 30%, and 40%. For the DS operator, we use mutation proportions of 20%, 40%, 60%, and 80%, as its impact on fine-tuning is relatively small. The fine-tuning process follows the same settings as described in Section 5.3. Finally, we determine the labels of the fine-tuned models in this section using the method outlined in Section 4.4 and count the number of faulty fine-tuned models.

The experimental results are presented in Fig. 7, where the x-axis represents the mutation proportion, and the y-axis represents the number of faulty models. As shown in Fig. 7, the number of faulty models increases with the mutation proportion for all mutation operators. For instance, the LE operator, which has the most significant impact, increases the number of faulty models from 93 at a 10% mutation proportion to 187 at a 50% mutation proportion. To ensure an adequate number of faulty samples for building classification models, we choose the mutation proportion of 50% (100% for DS) as the mutation proportion for our experiments.

7.5. Case study

Since pretraining and fine-tuning are two fundamentally different processes, their runtime information may inherently differ. That is, even under a non-faulty fine-tuning process, the runtime information can deviate from that obtained during pretraining. Therefore, it is necessary to investigate how such inherent differences affect the effectiveness of our method. To this end, we select a change metric⁴ as a case study to examine how our metric captures runtime information variations caused by data faults, and to explain why our approach is effective. Specifically, for each of the six dataset categories, we compute the change metric obtained by fine-tuning on non-faulty datasets and faulty⁵ datasets, respectively. We then use boxplots to visually illustrate the differences in their distribution ranges, and further calculate the corresponding whisker range and interquartile range (IQR).

⁴ In this section, we select *val_acc_euclidean_distance*, which measures the Euclidean distance between the validation accuracy sequences of the pretraining and fine-tuning processes.

⁵ In this section, we select fine-tuning datasets with label errors, since such errors have been observed to result in the largest proportion of faulty datasets.

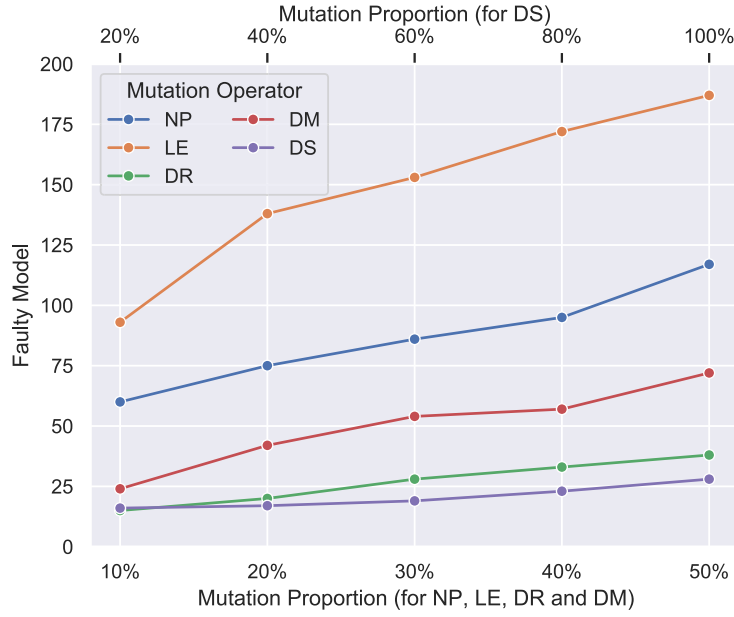


Fig. 7. The number of faulty fine-tuned models under different mutation proportions.

Table 10

Comparison of Whisker Range and IQR of Change Metric (val_acc_euclidean_distance) on Faulty and Non-Faulty Datasets.

	Whisker Range			IQR		
	Faulty	Non-Faulty	Ratio	Faulty	Non-Faulty	Ratio
Blob	0.935	0.225	416%	0.366	0.068	542%
Circle	0.527	0.241	219%	0.178	0.082	217%
MNIST	0.346	0.024	1418%	0.088	0.009	1023%
CIFAR-10	0.812	0.359	226%	0.230	0.119	194%
Reuters	1.484	0.134	1104%	0.826	0.037	2254%
IMDb	0.242	0.068	357%	0.188	0.028	672%

Fig. 8 presents a comparison of the boxplots, while Table 10 reports the whisker range and IQR, as well as the ratios of these statistics for faulty datasets relative to non-faulty ones. As shown, the change metric values obtained by fine-tuning on faulty datasets exhibit substantially larger whisker ranges and IQRs than those obtained from fine-tuning on non-faulty datasets. This difference is particularly pronounced on the Reuters dataset, where the IQR is $22\times$ larger. These results indicate that the variations induced by faulty datasets are significantly more prominent than those caused by other factors. Therefore, the proposed change metric can effectively distinguish and capture runtime information differences attributable to data faults.

In addition, we analyze the 37 change metrics in the top-50 gain ratio metrics from Section 7.1. For each proposed change metric, we compute the whisker range ratio and IQR ratio of the faulty datasets relative to the non-faulty datasets, and visualize the results using empirical cumulative distribution (ECDF) plots, as shown in Fig. 9. It can be observed that for most metrics (approximately 70% for CIFAR-10 and 80% for the other datasets), the whisker range ratios and the IQR ratios are greater than 1, with the whisker range ratio for MNIST reaching as high as 702. This indicates that the values of the change metrics are more widely distributed in the faulty datasets. Importantly, because these metrics are computed on runtime information that is directly affected by the fine-tuning data, the observed discrepancies can be attributed specifically to errors in the fine-tuning datasets. In other words, our change metrics are effective at distinguishing the differences caused by faulty data during fine-tuning.

7.6. The insights of our study

This paper introduces two types of change metrics, DSDM and MDCM, that capture anomalous differences between the pretraining and fine-tuning stages. Based on these metrics, we develop a classification model to diagnose data errors. We present five distinct mutation operators to ensure our approach supports five common types of dataset errors. Experimental results demonstrate that DSDM and MDCM surpass baseline methods in diagnosing data errors. Our study offers three insights for future work:

(a) *Monitoring differences from a change perspective is a practical and effective strategy for detecting potential data faults.* Our study indicates that even simple change metrics can provide meaningful signals about dataset errors. This highlights a more general

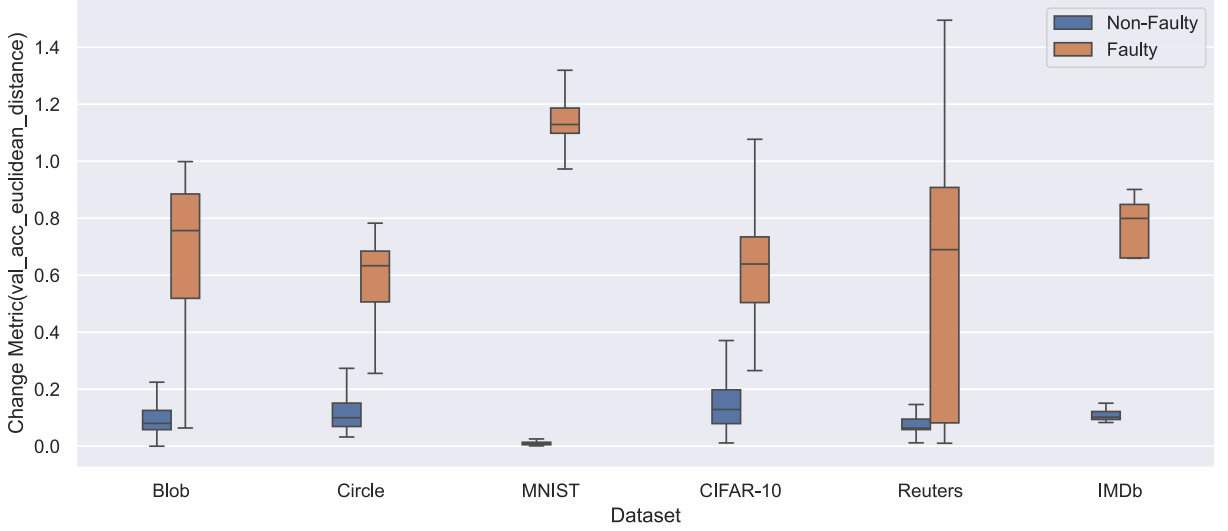


Fig. 8. Comparison of Change Metric (val_acc_euclidean_distance) on Faulty and Non-Faulty Datasets.

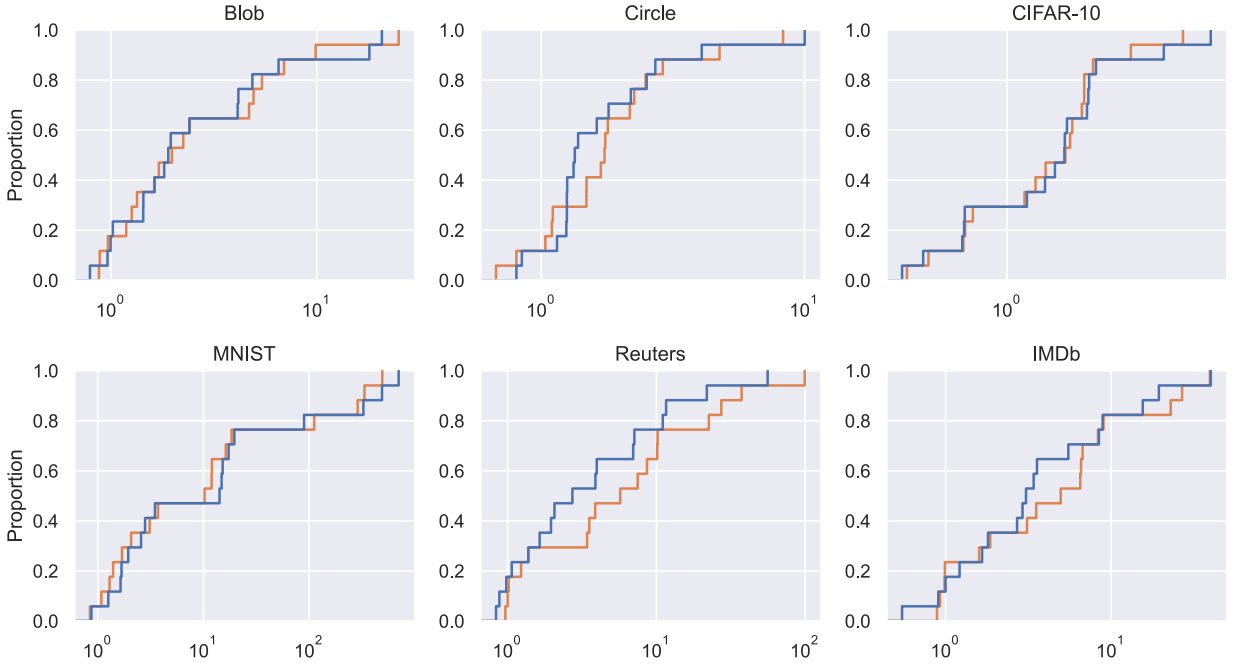


Fig. 9. ECDF of Whisker Range (■) and IQR (■) Ratios Across Different Datasets.

principle: in deep learning data debugging, relative changes in model behavior often reveal faults more effectively than absolute runtime information. Practitioners can thus adopt lightweight change metrics as an initial diagnostic tool, reducing reliance on complex task-specific metrics while still capturing essential fault signals.

(b) *Effective DL data debugging requires capturing the behavioral anomalies caused by data errors through multiple complementary change metrics.* A single metric can only reflect a limited aspect of how a faulty dataset affects model behavior. Employing multiple change metrics enables a more comprehensive characterization of these behavioral deviations. We suggest that practitioners should design diverse metrics to robustly capture the full spectrum of anomalies induced by real-world data errors.

(c) *Behavioral deviations induced by faulty fine-tuning dominate the inherent variations across different training stages.* Although natural differences exist between pretraining and fine-tuning, faults introduce more pronounced anomalies in model behavior. This demonstrates that change metrics are robust to benign variations and can serve as reliable indicators of abnormal behavior. Beyond data

error detection, this principle can be extended to identify other types of faults, such as issues in model architecture or inconsistencies in training procedures.

8. Threats to validity

In this section, we will discuss the threats to the validity of our work.

First, the selection and construction of datasets can be a threat. This paper selects six datasets containing multiple domains, such as points on a two-dimensional plane, images, and text, with varying levels of complexity. Although these datasets provide a diverse experimental basis, they may still cover only a limited portion of real-world scenarios, potentially restricting the generalizability of our findings. Regarding the construction of faulty fine-tuning datasets, we introduce Gaussian noise and adopt widely used mutation operators to simulate realistic fine-tuning faults. However, they cannot fully capture the diversity and subtlety of real-world data issues. Therefore, the generalizability of the proposed change metrics to practical scenarios requires further investigation.

Second, the selection of DL models and training configurations may also pose a threat to validity. Although we evaluate our approach on 233 standard DL models with diverse architectures and accuracy levels, following the hyperparameter settings adopted in prior studies [12,27], runtime behaviors may still be influenced by factors such as learning rate schedules, optimization strategies, and random initialization. Such variations may introduce additional deviations that are not directly caused by faulty fine-tuning data.

Finally, from a practical perspective, the proposed approach relies on collecting runtime information throughout different training stages, which inevitably introduces additional computational overhead. Therefore, the overall time cost is largely tied to the training process itself, particularly for large-scale models or datasets with long training cycles, which may limit scalability in real-world deployment, especially in resource-constrained environments where training efficiency is a primary concern. Future work could explore more efficient monitoring strategies or lightweight approximations to reduce the practical overhead.

9. Conclusion

In this study, we propose two aspects of the change metrics: Descriptive Statistics Difference Metrics (DSDM) and Multidimensional Data Comparison Metrics (MDCM) to diagnose data bugs in the fine-tuning process. The proposed metrics are calculated by runtime data collected during pretraining and fine-tuning. We evaluate the power of our change metrics on five common classifiers. The results indicate that (a) our change metrics are more effective than the baseline in fault diagnosis; (b) in most cases, using DSDM and MDCM together is more effective than using them separately; (c) the time cost of our approach primarily depends on the model training process, which is generally acceptable overall.

9.1. Replication package

We provide data and source code used to conduct this study at <https://github.com/ChangeMetric/CM>.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used Grammarly and ChatGPT to enhance the readability and language of the manuscript. After utilizing these tools, the authors reviewed and edited the content as necessary and take full responsibility for the final version of the published article.

CRediT authorship contribution statement

Wenjie Xu: Writing – review & editing, Writing – original draft, Methodology, Data curation; **Yanhui Li:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition; **Mingliang Ma:** Methodology, Data curation; **Lin Chen:** Writing – review & editing; **Yuming Zhou:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank anonymous reviewers for their time and invaluable feedback to improve our study. This work is supported by the [National Natural Science Foundation of China](#) (Grant Nos. 62172202, 62272221, and 62572226).

References

- [1] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, *Commun. ACM* 60 (6) (2017) 84–90. <https://doi.org/10.1145/3065386>
- [2] P. He, C. Meister, Z. Su, Structure-invariant testing for machine translation, in: G. Rothermel, D. Bae (Eds.), *ICSE '20: 42nd International Conference on Software Engineering*, Seoul, South Korea, 27 June - 19 July, 2020, ACM, 2020, pp. 961–973. <https://doi.org/10.1145/3377811.3380339>
- [3] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444.
- [4] Y. Liu, G. Liu, Q. Zhang, Deep learning and medical diagnosis, *Lancet* 394 (10210) (2019) 1709–1710.
- [5] K. Muhammad, A. Ullah, J. Lloret, J. Del Ser, V.H.C. de Albuquerque, Deep learning for safe autonomous driving: current challenges and future directions, *IEEE Trans. Intell. Transp. Syst.* 22 (7) (2020) 4316–4336.
- [6] A.M. Ozbayoglu, M.U. Gudelek, O.B. Sezer, Deep learning for financial applications: a survey, *Appl. Soft Comput.* 93 (2020) 106384.
- [7] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, P. Tonella, Taxonomy of real faults in deep learning systems, in: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1110–1121.
- [8] L. Meng, Y. Li, L. Chen, Z. Wang, D. Wu, Y. Zhou, B. Xu, Measuring discrimination to boost comparative testing for multiple deep learning models, in: *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, IEEE, 2021, pp. 385–396.
- [9] Q. Guo, S. Chen, X. Xie, L. Ma, Q. Hu, H. Liu, Y. Liu, J. Zhao, X. Li, An empirical study towards characterizing deep learning development and deployment across different frameworks and platforms, in: *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2019, pp. 810–822.
- [10] J.M. Zhang, M. Harman, L. Ma, Y. Liu, Machine learning testing: survey, landscapes and horizons, *IEEE Trans. Softw. Eng.* 48 (1) (2020) 1–36.
- [11] N. Polyzotis, M. Zinkevich, S. Roy, E. Breck, S. Whang, Data validation for machine learning, *Proc. Mach. Learn. Syst.* 1 (2019) 334–347.
- [12] J. Cao, M. Li, X. Chen, M. Wen, Y. Tian, B. Wu, S.-C. Cheung, DeepFD: automated fault diagnosis and localization for deep learning programs, in: *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 573–585.
- [13] H. Zhang, W.K. Chan, Apricot: a weight-adaptation approach to fixing deep learning models, in: *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11–15, 2019*, IEEE, 2019, pp. 376–387. <https://doi.org/10.1109/ASE.2019.00043>
- [14] A. Halevy, P. Norvig, F. Pereira, The unreasonable effectiveness of data, *IEEE Intell. Syst.* 24 (2) (2009) 8–12.
- [15] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. Paritosh, L.M. Aroyo, “Everyone wants to do the model work, not the data work”: data cascades in high-stakes AI, in: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021, pp. 1–15.
- [16] O. Nahum, N. Calderon, O. Keller, I. Szpektor, R. Reichart, Are LLMs better than reported? Detecting label errors and mitigating their effect on model performance, *arXiv:2410.18889* (2024).
- [17] W. Liang, G.A. Tadesse, D. Ho, L. Fei-Fei, M. Zaharia, C. Zhang, J. Zou, Advances, challenges and opportunities in creating data for trustworthy AI, *Nat. Mach. Intell.* 4 (8) (2022) 669–677.
- [18] Y. Yin, Y. Feng, S. Weng, Z. Liu, Y. Yao, Y. Zhang, Z. Zhao, Z. Chen, Dynamic data fault localization for deep neural networks, in: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1345–1357.
- [19] E. Strickland, IBM Watson, heal thyself: how IBM overpromised and underdelivered on AI health care, *IEEE Spectr.* 56 (4) (2019) 24–31.
- [20] Y. Li, L. Meng, L. Chen, L. Yu, D. Wu, Y. Zhou, B. Xu, Training data debugging for the fairness of machine learning software, in: *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2215–2227.
- [21] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, T. Zimmermann, Software engineering for machine learning: a case study, in: *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, IEEE, 2019, pp. 291–300.
- [22] Q. Hu, Y. Guo, M. Cordy, X. Xie, L. Ma, M. Papadakis, Y. Le Traon, An empirical study on data distribution-aware test selection for deep learning enhancement, *ACM Trans. Softw. Eng. Methodol.* 31 (4) (2022) 1–30.
- [23] D. Berend, X. Xie, L. Ma, L. Zhou, Y. Liu, C. Xu, J. Zhao, Cats are not fish: deep learning testing calls for out-of-distribution awareness, in: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 1041–1052.
- [24] W. Shen, Y. Li, L. Chen, Y. Han, Y. Zhou, B. Xu, Multiple-boundary clustering and prioritization to promote neural network retraining, in: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 410–422.
- [25] F. García, M.F. Bertoa, C. Calero, A. Vallecillo, F. Ruiz, M. Piattini, M. Genero, Towards a consistent terminology for software measurement, *Inf. Softw. Technol.* 48 (8) (2006) 631–644.
- [26] R. Moser, W. Pedrycz, G. Succì, A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction, in: *Proceedings of the 30th International Conference on Software Engineering*, 2008, pp. 181–190.
- [27] X. Zhang, J. Zhai, S. Ma, C. Shen, Autotrainer: an automatic dnn training problem detection and repair system, in: *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, IEEE, 2021, pp. 359–371.
- [28] L. Ma, F. Zhang, J. Sun, M. Xue, B. Li, F. Juefei-Xu, C. Xie, L. Li, Y. Liu, J. Zhao, et al., Deepmutation: mutation testing of deep learning systems, in: *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*, IEEE, 2018, pp. 100–111.
- [29] W. Shen, Y. Li, Y. Han, L. Chen, D. Wu, Y. Zhou, B. Xu, Boundary sampling to boost mutation testing for deep learning models, *Inf. Softw. Technol.* 130 (2021) 106413.
- [30] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, L. Zhang, An empirical study on tensorflow program bugs, in: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2018, pp. 129–140.
- [31] M.J. Islam, G. Nguyen, R. Pan, H. Rajan, A comprehensive study on deep learning bug characteristics, in: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 510–520.
- [32] M.M. Morovati, A. Nikanjam, F. Tambon, F. Khomh, Z. Ming, et al., Bug characterization in machine learning-based systems, *arXiv:2307.14512* (2023).
- [33] L. Jia, H. Zhong, X. Wang, L. Huang, X. Lu, An empirical study on bugs inside tensorflow, in: *Database Systems for Advanced Applications: 25th International Conference, DASFAA 2020, Jeju, South Korea, September 24–27, 2020, Proceedings, Part I 25*, Springer, 2020, pp. 604–620.
- [34] L. Jia, H. Zhong, X. Wang, L. Huang, X. Lu, The symptoms, causes, and repairs of bugs inside a deep learning library, *J. Syst. Softw.* 177 (2021) 110935.
- [35] J. Chen, Y. Liang, Q. Shen, J. Jiang, S. Li, Toward understanding deep learning framework bugs, *ACM Trans. Softw. Eng. Method.* 32(6) (2023) 1–31.
- [36] M. Wardat, W. Le, H. Rajan, Deeplocalize: fault localization for deep neural networks, in: *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, IEEE, 2021, pp. 251–262.
- [37] E. Schoop, F. Huang, B. Hartmann, Umlaut: debugging deep learning programs using program structure and model behavior, in: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021, pp. 1–16.
- [38] A. Nikanjam, H.B. Braiek, M.M. Morovati, F. Khomh, Automatic fault detection for deep learning programs using graph transformations, *ACM Trans. Softw. Eng. Methodol.* 31 (1) (2021) 1–27.
- [39] M. Wardat, B.D. Cruz, W. Le, H. Rajan, DeepDiagnosis: automatically diagnosing faults and recommending actionable fixes in deep learning programs, in: *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 561–572.
- [40] M. Wardat, B.D. Cruz, W. Le, H. Rajan, An effective data-driven approach for localizing deep learning faults, *arXiv:2307.08947* (2023).
- [41] Y. Tian, K. Pei, S. Jana, B. Ray, Deeptest: automated testing of deep-neural-network-driven autonomous cars, in: *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 303–314.
- [42] M. Zhang, Y. Zhang, L. Zhang, C. Liu, S. Khurshid, DeepRoad: GAN-based metamorphic testing and input validation framework for autonomous driving systems, in: *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 132–142.
- [43] H. Zhou, W. Li, Z. Kong, J. Guo, Y. Zhang, B. Yu, L. Zhang, C. Liu, Deepbillboard: systematic physical-world testing of autonomous driving systems, in: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 347–358.
- [44] A. Guo, Y. Feng, Z. Chen, LiRTTest: augmenting LiDAR point clouds for automated testing of autonomous driving systems, in: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 480–492.

- [45] Z. Sun, J.M. Zhang, M. Harman, M. Papadakis, L. Zhang, Automatic testing and improvement of machine translation, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, 2020, pp. 974–985.
- [46] S. Gupta, P. He, C. Meister, Z. Su, Machine translation testing via pathological invariance, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2020, pp. 863–875.
- [47] K. Pei, Y. Cao, J. Yang, S. Jana, DeepXplore: automated whitebox testing of deep learning systems, in: Proceedings of the 26th Symposium on Operating Systems Principles, 2017, pp. 1–18.
- [48] X. Xie, L. Ma, F. Juefei-Xu, M. Xue, H. Chen, Y. Liu, J. Zhao, B. Li, J. Yin, S. See, DeepHunter: a coverage-guided fuzz testing framework for deep neural networks, in: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2019, pp. 146–157.
- [49] S. Lee, S. Cha, D. Lee, H. Oh, Effective white-box testing of deep neural networks with adaptive neuron-selection strategy, in: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2020, pp. 165–176.
- [50] J. Wang, H. Qiu, Y. Rong, H. Ye, Q. Li, Z. Li, C. Zhang, BET: black-box efficient testing for convolutional neural networks, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 164–175.
- [51] J. Yu, S. Duan, X. Ye, A white-box testing for deep neural networks based on neuron coverage, *IEEE Trans. Neural Netw. Learn. Syst.* 34(11) (2022) 9185–9197.
- [52] S. Kim, E.J. Whitehead, Y. Zhang, Classifying software changes: clean or buggy?, *IEEE Trans. Softw. Eng.* 34 (2) (2008) 181–196.
- [53] L. Šikić, P. Afrić, A.S. Kurdija, M. Šilić, Improving software defect prediction by aggregated change metrics, *IEEE Access* 9 (2021) 19391–19411.
- [54] G.R. Choudhary, S. Kumar, K. Kumar, A. Mishra, C. Catal, Empirical analysis of change metrics for software fault prediction, *Comput. Electr. Eng.* 67 (2018) 15–24.
- [55] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324.
- [56] A. Krizhevsky, G. Hinton, et al., Learning multiple layers of features from tiny images (2009) <http://www.cs.utoronto.ca/~kriz/learning-features-2009-TR.pdf>.
- [57] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al., Imagenet large scale visual recognition challenge, *Int. J. Comput. Vis.* 115 (2015) 211–252.
- [58] S. Gupta, A. Gupta, Dealing with noise problem in machine learning data-sets: a systematic review, *Procedia Comput. Sci.* 161 (2019) 466–474.
- [59] N. Rauschmayr, V. Kumar, R. Huilgol, A. Olgiati, S. Bhattacharjee, N. Harish, V. Kannan, A. Lele, A. Acharya, J. Nielsen, et al., Amazon SageMaker debugger: a system for real-time insights into machine learning model training, *Proc. Mach. Learn. Syst.* 3 (2021) 770–782.
- [60] K. An, Sulla determinazione empirica di una legge di distribuzione, *Giorn. Dell'inst. Ital. Degli Atti* 4 (1933) 89–91.
- [61] A. Gretton, K.M. Borgwardt, M.J. Rasch, B. Schölkopf, A. Smola, A kernel two-sample test, *J. Mach. Learn. Res.* 13 (1) (2012) 723–773.
- [62] N. Humbatova, G. Jahangirova, P. Tonella, DeepPrime: mutation testing of deep learning systems based on real faults, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 67–78.
- [63] G. Jahangirova, P. Tonella, An empirical evaluation of mutation operators for deep learning systems, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), IEEE, 2020, pp. 74–84.
- [64] J.A. Nelder, R.W.M. Wedderburn, Generalized linear models, *J. R. Stat. Soc. Ser. A Stat. Soc.* 135 (3) (1972) 370–384.
- [65] J. Cohen, A power primer, *Psychol. Bull.* 112 (1992) 155–159.
- [66] J.R. Quinlan, Induction of decision trees, *Mach. Learn.* 1 (1986) 81–106.
- [67] L. Breiman, Random forests, *Mach. Learn.* 45 (2001) 5–32.
- [68] T. Cover, P. Hart, Nearest neighbor pattern classification, *IEEE Trans. Inf. Theory* 13 (1) (1967) 21–27.
- [69] scikit-learn, Decision Trees, 2025, <https://scikit-learn.org/stable/modules/tree.html>.
- [70] W. contributors, Random forest – Wikipedia, 2025, [Online; accessed 26-February-2025], https://en.wikipedia.org/wiki/Random_forest.
- [71] Wikipedia contributors, Naive Bayes classifier – Wikipedia, the free encyclopedia, 2025, (2025). [Online; accessed 26-February-2025], https://en.wikipedia.org/wiki/Naive_Bayes_classifier.
- [72] Wikipedia contributors, K-nearest neighbors algorithm — Wikipedia, the free encyclopedia, 2025, (2025). [Online; accessed 26-February-2025], https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm.
- [73] scikit-learn, Linear models, 2025, https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.
- [74] scikit-learn, Sklearn, make circles dataset, 2025, (2025). https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_circles.html.
- [75] scikit-learn, Sklearn, make blobs dataset, 2025, (2025). https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_blobs.html.
- [76] Y. LeCun, C. Corinna, MNIST handwritten digit database, 2010, (2010). <http://yann.lecun.com/exdb/mnist/>.
- [77] A. Maas, R.E. Daly, P.T. Pham, D. Huang, A.Y. Ng, C. Potts, Learning word vectors for sentiment analysis, in: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, 2011, pp. 142–150.
- [78] D. Lewis, Reuters-21578 text categorization collection, 1997, (UCI Machine Learning Repository). <https://doi.org/10.24432/C52G6M>.
- [79] scikit-learn, machine learning in python, 2025, <https://scikit-learn.org/stable/>.
- [80] F. Wilcoxon, Individual comparisons by ranking methods, in: Breakthroughs in Statistics: Methodology and Distribution, Springer, 1992, pp. 196–202.
- [81] J. Romano, J.D. Kromrey, J. Coraggio, J. Skowronek, L. Devine, Exploring methods for evaluating group differences on the NSSE and other surveys: are the t-test and Cohen's d indices the most appropriate choices, in: Annual Meeting of the Southern Association for Institutional Research, 2006.
- [82] J. Nam, S. Kim, Heterogeneous defect prediction, in: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, 2015, pp. 508–519.
- [83] Y. Liu, Y. Li, J. Guo, Y. Zhou, B. Xu, Connecting software metrics across versions to predict defects, in: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2018, pp. 232–243.
- [84] S. Shivaji, E.J. Whitehead, R. Akella, S. Kim, Reducing features to improve bug prediction, in: 2009 IEEE/ACM International Conference on Automated Software Engineering, IEEE, 2009, pp. 600–604.
- [85] D. Kim, X. Wang, S. Kim, A. Zeller, S.-C. Cheung, S. Park, Which crashes should i fix first?: predicting top crashes at an early stage to prioritize debugging efforts, *IEEE Trans. Softw. Eng.* 37 (3) (2011) 430–447.
- [86] T. Lee, J. Nam, D. Han, S. Kim, H.P. In, Developer micro interaction metrics for software defect prediction, *IEEE Trans. Softw. Eng.* 42 (11) (2016) 1015–1035.
- [87] Z. Xu, J. Liu, X. Luo, Z. Yang, Y. Zhang, P. Yuan, Y. Tang, T. Zhang, Software defect prediction based on kernel PCA and weighted extreme learning machine, *Inf. Softw. Technol.* 106 (2019) 182–200.